Presence Protocol: Distributed Proximity Verification Architecture

Document Type: Protocol Design & Engineering Reference
Version: 1.0
Scope: Encounter Proof Layer, Anti-Fraud Systems, BLE Network Design, Economic Model

---

Executive Summary

Presence Protocol aims to build a cryptographically verifiable global proximity network on mobile devices. The core engineering challenges are not novel in isolation — BLE mesh networking, zero-knowledge proofs, and token economics are individually well-understood — but combining them at global scale on resource-constrained consumer hardware, against an adversary with financial incentive to cheat, is genuinely hard.

This document provides concrete protocol designs, data structures, and engineering strategies for each major challenge, drawing on lessons from Nodle, Helium, and POAP. The central thesis is: **the encounter proof itself must be the primary fraud boundary**. Every other layer — relay rewards, collusion detection, economic policy — is defense-in-depth. A weak proof structure makes all other defenses moot.

Lessons from Comparable Projects

Nodle

What they got right: BLE scanning offloaded to a background SDK; separation of scanning duty from proof submission; operator staking for relay nodes.
What failed: SDK-level Sybil attacks by app farms; lack of cryptographic binding between device identity and BLE observation; reward inflation from unverified scan claims.

Helium (Proof-of-Coverage)

What they got right: Witness-based coverage proofs; beaconer/challenger separation; staking to gate mining entry.
What failed: Spoofed GPS coordinates allowing geographic fraud; gaming via VPN/SDR setups; "gaming islands" — small colluding clusters that verified each other in isolated geographic pockets; severe token inflation during growth phase.

POAP

What they got right: Simple NFT-based attendance proof with event-scoped codes.
What failed: Code sharing via social media (trivially replayable); no physical presence verification beyond code knowledge; bulk minting by bots via leaked codes.

Common Failure Modes (Synthesized)

1. Proofs that are separable from the prover (codes, keys, or signals that can be forwarded)
2. Identity that is cheap to create and reset

3. Reward mechanisms with no supply ceiling or difficulty adjustment
4. Relay/witness roles with no stake or accountability
5. Proofs that require trust in a central party for validity

Part I: Cryptographic Encounter Proof Design

1.1 Core Proof Structure

An encounter proof must be **mutually constructed** — neither party can produce it alone, and the construction process must be unforgeable even if one party is malicious.

```
EncounterProof {
  // Identity layer (pseudonymous, rotatable)
  ephemeral_id_A:     bytes[32]   // Rotating pseudonymous ID for device A
  ephemeral_id_B:     bytes[32]   // Rotating pseudonymous ID for device B
  epoch:              uint32      // Time epoch (e.g. 15-minute windows)

  // Binding layer (prevents replay, links to hardware)
  nonce_A:            bytes[32]   // Fresh random nonce from A
  nonce_B:            bytes[32]   // Fresh random nonce from B
  session_id:         bytes[16]   // Derived: H(nonce_A || nonce_B || epoch)

  // Signal layer (hard to fake without physical proximity)
  rssi_A_to_B:        int8        // Signal strength observed by A
  rssi_B_to_A:        int8        // Signal strength observed by B
  tx_power_A:         int8        // Advertised TX power of A
  tx_power_B:         int8        // Advertised TX power of B
  channel_sequence:   bytes[4]    // BLE channel hopping sequence fingerprint
  packet_timing_ns:   uint64[]    // Packet arrival timestamps (hardware-level)

  // Attestation layer
  sig_A:              bytes[64]   // A signs: H(session_id || rssi_B_to_A || nonce_B)
  sig_B:              bytes[64]   // B signs: H(session_id || rssi_A_to_B || nonce_A)

  // Auxiliary
  duration_ms:        uint32      // Encounter duration
  proof_version:      uint8       // Protocol version
  hardware_attestation_A: bytes[] // Optional: Android SafetyNet/Play Integrity token
  hardware_attestation_B: bytes[] // Optional: Android SafetyNet/Play Integrity token
}
```

Why this structure is robust:

- nonce_A and nonce_B are exchanged during the GATT handshake. Neither party knows the other's nonce until the handshake happens in real time. Recording and replaying requires replaying the nonces, which requires the other party's live participation.
- session_id = H(nonce_A || nonce_B || epoch) binds the proof to a specific moment. If the epoch window has passed, the session_id is invalid against any on-chain epoch commitment.
- Each party signs the *other's* nonce bound to their *own* observed RSSI. A cannot forge B's signature, and B cannot forge A's RSSI observation without controlling A's hardware.
- RSSI values from both directions must be internally consistent (bidirectional signal budget), which is difficult to fake without real hardware at real distances.

1.2 GATT Handshake Protocol

The handshake runs over BLE GATT after discovery. Device A discovers B via BLE advertising.

```
Phase 1: Initialization
──────────────────────────────────────

A → B   [GATT WRITE]  nonce_A (32 bytes), epoch, ephemeral_id_A
B → A   [GATT NOTIFY] nonce_B (32 bytes), ephemeral_id_B

Phase 2: Mutual Commitment
──────────────────────────────────────────

A computes: session_id = H(nonce_A || nonce_B || epoch)
B computes: session_id = H(nonce_A || nonce_B || epoch)

Phase 3: Signature Exchange
──────────────────────────────────────────────

A → B   [GATT WRITE]  sig_A = Sign_A(H(session_id || rssi_B_to_A || nonce_B))
B → A   [GATT NOTIFY] sig_B = Sign_B(H(session_id || rssi_A_to_B || nonce_A))

Phase 4: Proof Assembly
──────────────────────────────────────

Both devices locally assemble the full EncounterProof.
Neither needs to be online. Proof is stored locally for later submission.
```

Critical implementation notes:

- Use Ed25519 for signatures (fast, small, widely supported on Android)
- The private key for signing must be generated inside Android Keystore (hardware-backed on most modern devices) and never exported. This ties signatures to a specific hardware instance.
- Phase 3 must complete within a timeout (e.g. 5 seconds). Incomplete handshakes are discarded.
- GATT MTU negotiation should be performed first; target 185 bytes MTU to fit all handshake data in minimal packets.

1.3 Ephemeral Identity Rotation

Devices should not use persistent identifiers — that would allow tracking. Instead:

```
// Every 15-minute epoch:
epoch_seed = HKDF(master_secret, epoch_number, "presence-eid-v1")
ephemeral_id = epoch_seed[0:32]

// The master_secret is registered on-chain via a commitment:
commitment = H(master_secret || device_salt)
```

This is similar to the Google/Apple Exposure Notification scheme. The ephemeral_id rotates, so observers cannot link encounters across epochs. But because master_secret is committed on-chain, valid encounters can later be attributed to the correct device for reward purposes — without revealing the master_secret publicly.

Reward attribution flow:

1. Device submits proof on-chain, reveals the epoch_seed for that window only.
2. Verifier checks: `HKDF(committed_master_secret, epoch) == epoch_seed` (in ZK).
3. Reward is issued to the committed device identity without exposing master_secret.

This requires a ZK proof of knowledge of master_secret given the commitment and epoch_seed. Circom/SnarkJS or Noir (for Midnight integration) can implement this in a mobile-feasible proof size.

Part II: Sybil Resistance

2.1 Hardware Attestation as the First Gate

The most effective Sybil resistance is binding device identity to physical hardware. Android provides two mechanisms:

Android Play Integrity API (preferred for 2024+):

- Returns a signed token from Google attesting: this is a genuine Android device, the app is unmodified, and the device has not been rooted.
- Token is tied to a nonce you supply — bind it to the session_id from the handshake.
- Cannot be generated by emulators or rooted devices running hooking frameworks.
- Token is timestamped and expires — it cannot be pre-generated and stockpiled.

```kotlin
```

```
// Request integrity verdict bound to handshake session
val integrityManager = IntegrityManagerFactory.create(context)
val request = StandardIntegrityTokenRequest.builder()
    .setRequestHash(sessionId.toBase64())  // Bind to handshake
    .build()
val token = integrityManager.requestIntegrityToken(request).await()
// Include token in hardware_attestation field of EncounterProof
```

Limitations to design around:

- Play Integrity requires Google Play Services — unavailable on some Android variants (GrapheneOS, Chinese OEMs without GMS).
- Tokens have a quota per app per device per day — design submission batching accordingly.
- Attestation verifies the *device* but not the *location*. It rules out emulators but not device farms of real phones.

Supplementary Sybil signals:

- Accelerometer data during encounter window (stationary devices in farms have distinctive flat motion signatures)
- Wi-Fi and cellular tower observation counts (farms typically have homogeneous radio environments)
- Diversity of encounter graph over time (Sybil clusters encounter each other repeatedly — see Collusion Detection)

2.2 Staking and Reputation

Economic Sybil resistance: make fake identities costly.

```
Device Registration:
  - On first use, device generates master_secret, computes commitment
  - To earn rewards, device must stake minimum_stake tokens
  - Stake is slashable on proven fraud detection
  - New devices earn at reduced rate (probationary period: first 30 days or 100 encounters)
  - Stake amount scales reward multiplier: stake_tier ∈ {bronze, silver, gold}

Probationary Mechanics:
  - Encounters during probation stored but rewards held in escrow
  - After probation, retrospective review of encounter graph
  - If graph looks suspicious, escrowed rewards slashed
  - This delay creates a capital lockup cost for Sybil operators
```

2.3 Physical Unclonable Functions (Future)

As hardware matures, consider integrating with device TEE (Trusted Execution Environment) attestation keys

that are provisioned at manufacturing. Qualcomm Snapdragon and MediaTek both expose TEE-based key generation that cannot be extracted even with root access. This is a stronger Sybil resistance mechanism than Play Integrity for high-value use cases.

Part III: Replay Attack Prevention

3.1 Why Temporal Binding Matters

A replay attack means: record a legitimate encounter proof, then submit it again later (or submit it in multiple geographic regions to claim impossible encounters). The handshake design in Section 1.2 makes replay hard at the proof-generation level. Additional defenses are needed at the submission level.

3.2 Epoch-Bounded Proofs

```
Proof validity window:
  - Each epoch = 15 minutes (configurable per network version)
  - Epoch number = floor(unix_timestamp / 900)
  - Proof is only valid for submission within 24 hours of its epoch
  - After 24 hours, the proof's epoch_seed commitment has expired
  - Network tracks submitted (session_id, epoch) pairs to detect duplicates

On-chain replay detection:
  - session_id = H(nonce_A || nonce_B || epoch) is submitted with proof
  - Network maintains a Bloom filter (or sparse Merkle tree) of seen session_ids
  - Duplicate session_id submissions are rejected and may trigger slash
```

The 24-hour submission window balances two constraints: users may be offline (e.g. in areas with no connectivity) and need time to submit, but infinite windows allow stockpiling proofs.

3.3 Nonce Uniqueness Enforcement

```
nonce_A is generated as:
  nonce_A = H(device_secret || timestamp_ms || crypto_random_bytes(16))

Properties:
  - timestamp_ms makes it unique per millisecond
  - crypto_random_bytes provides additional entropy even if clock is wrong
  - device_secret means a different device cannot predict or replay A's nonces
  - Network can verify nonce freshness via the epoch timestamp field
```

3.4 Multi-submission Detection

A single encounter should produce one proof. If the same (device_A, device_B, epoch) tuple appears multiple times with different session_ids, that is suspicious:

- Could indicate a software bug (duplicate submission)
- Could indicate an attempt to submit multiple proofs from the same physical encounter (reward farming)

Policy: only the first valid proof per (device_A, device_B, epoch) is rewarded. Additional submissions are recorded for fraud analysis but receive no reward.

Part IV: Collusion Detection

4.1 The Graph Structure of Fraud

Colluding devices have a distinctive graph signature: a small, densely connected clique that encounters each other far more frequently than any organic proximity network would produce. Organic encounter graphs follow power-law distributions — most device pairs encounter each other rarely, a few pairs encounter each other occasionally.

4.2 Encounter Graph Analysis

Maintain a directed weighted graph G = (V, E) where:

- V = device identities (ephemeral, unlinkable across epochs for privacy, but linkable within an epoch window for fraud detection)
- E = encounter counts between device pairs
- Edge weight = number of encounters in rolling 7-day window

Red flags:

```
Clique density:
  - A group of N devices with >80% of possible N*(N-1)/2 edges present
  - Normal social encounter graphs have <5% density for groups >10

Edge weight outliers:
  - A pair with >10x the median encounters per day
  - Median daily encounter rate in dense urban areas: ~15-30 unique encounters
  - A suspicious pair: 500+ encounters per day (impossible without co-location loop)

Geographic impossibility:
  - Device A's encounter graph implies it was in two places simultaneously
  - Detect via encounter partner location estimation (triangulation from BLE signal paths)

Temporal clustering:
  - All encounters for a suspicious group occur in bursts at identical times
  - Organic encounters have organic timing distributions; farms run on scripts
```

4.3 Anomaly Scoring Pipeline

```
For each device D, compute daily fraud score:

  F(D) = w1 * clique_score(D)
      + w2 * edge_concentration(D)    // What fraction of encounters are with same partners?
      + w3 * timing_entropy(D)        // Low entropy = scripted behavior
      + w4 * rssi_variance(D)         // Too-consistent RSSI = stationary farm
      + w5 * acceleration_correlation(D)  // Motion synchronized with partners?

Where weights w1..w5 are calibrated on known fraud samples.

Action thresholds:
  F(D) > 0.7: Flag for review, hold rewards in escrow
  F(D) > 0.9: Suspend reward accumulation, trigger community challenge
  Confirmed fraud: Slash stake, blacklist device commitment
```

4.4 Stake-Weighted Reporting

Community members can submit fraud evidence against a device, staking their own tokens on the claim. If the fraud is confirmed, the reporter earns a share of the slashed stake. If the report is rejected, the reporter's stake is slashed. This aligns incentives for honest fraud reporting without requiring a central fraud team.

Part V: Battery Efficiency

5.1 The Core BLE Power Problem

Continuous BLE scanning is the biggest battery drain in proximity networks. The fundamental parameters are:

```
BLE Scan Modes (Android):
  SCAN_MODE_LOW_POWER:      ~1.5 mA, scan 0.5s every 5s
  SCAN_MODE_BALANCED:       ~7 mA,   scan 2s every 5s
  SCAN_MODE_LOW_LATENCY:    ~20 mA,  continuous scan
  SCAN_MODE_OPPORTUNISTIC:  ~0 mA,   piggybacks on other app scans

BLE Advertising:
  Advertising interval: 100ms–10240ms
  Current draw: ~10 mA during advertisement burst
  Background advertising on Android: possible via BLE peripheral mode
```

5.2 Adaptive Duty-Cycle Strategy

Do not use a fixed scan mode. Use context signals to adjust dynamically:

```
kotlin
```

```kotlin
enum class ScanIntensity { MINIMAL, REDUCED, NORMAL, ELEVATED }

fun computeScanIntensity(context: DeviceContext): ScanIntensity {
    return when {
        context.isCharging -> ELEVATED          // Free power, scan more
        context.batteryLevel < 15 -> MINIMAL    // Emergency conservation
        context.isStationary -> REDUCED         // Not moving, fewer new encounters expected
        context.isInKnownHighDensityArea -> NORMAL  // Opportunistic
        context.recentEncounterCount > 5 -> REDUCED // Already got proofs this window
        else -> NORMAL
    }
}

// Map to Android scan parameters
fun ScanIntensity.toScanSettings(): ScanSettings = when (this) {
    MINIMAL  -> ScanSettings.SCAN_MODE_LOW_POWER with 15-second intervals
    REDUCED  -> ScanSettings.SCAN_MODE_LOW_POWER with 5-second intervals
    NORMAL   -> ScanSettings.SCAN_MODE_BALANCED
    ELEVATED -> ScanSettings.SCAN_MODE_LOW_LATENCY
}
```

5.3 Advertising Interval Optimization

Use longer advertising intervals by default; shorten them only when actively seeking to complete a handshake:

```
Advertising states:
  IDLE (no nearby devices recently seen):
    → Interval: 2000ms, Advertise for 200ms per burst
    → Current drain: ~1 mA average

  DISCOVERY (recently saw a new device):
    → Interval: 200ms, Advertise for 30ms per burst
    → Hold for 10 seconds to complete handshake, then return to IDLE

  HANDSHAKE (GATT connection in progress):
    → Advertising paused (BLE peripheral mode occupied)
    → Total handshake window: <3 seconds target
```

5.4 Opportunistic Scanning via JobScheduler

For background operation when the screen is off:

```
kotlin
```

```
// Use WorkManager with network/power constraints
val encounterWork = PeriodicWorkRequestBuilder<EncounterScanWorker>(
    repeatInterval = 15,
    repeatIntervalTimeUnit = TimeUnit.MINUTES
)
.setConstraints(
    Constraints.Builder()
        .setRequiresBatteryNotLow(true)
        .build()
)
.build()

// Also register for BLE scan results from other apps via
// SCAN_MODE_OPPORTUNISTIC — zero battery cost, captures results
// from system or other app BLE scans
```

5.5 Batched GATT Handshakes

If multiple devices are nearby, serialize handshakes rather than running them in parallel. Each GATT connection requires radio time-sharing. Parallel connections cause retransmissions and increased power draw. Queue up to 3 pending handshakes and process them sequentially.

5.6 Expected Power Budget

With the above strategy on a typical 4500 mAh Android device:

Mode	Estimated Daily Drain
No Presence Protocol	30-40% (baseline)
+ Low-power scan, low density	+3-5%
+ Balanced scan, urban	+8-12%
+ Elevated (charging)	+20-30% (acceptable, charging)

Target: Less than 10% additional battery drain in normal urban use. User surveys from Nodle suggest >12% additional drain causes significant churn.

Part VI: Privacy Architecture

6.1 Privacy Threat Model

The privacy threat model for a proximity network has three distinct adversaries:

1. **The curious server/network:** The infrastructure operator should not learn who you met, when, or where.

2. **A colluding encounter partner:** Device B should not learn Device A's persistent identity.
3. **A passive observer:** A third party scanning BLE advertisements should not be able to track Device A over time.

6.2 Ephemeral Pseudonymity

As described in Section 1.3, rotating ephemeral IDs every 15 minutes defeats passive observers. This is the same mechanism used in COVID exposure notification systems. The key addition for Presence Protocol is that reward attribution must work without breaking this privacy guarantee.

6.3 Zero-Knowledge Reward Attribution

The flow works as follows:

```
Off-chain (on device):
  master_secret  →  [HKDF + epoch]  →  ephemeral_id (public during encounter)

  Device generates ZK proof π such that:
    π proves: "I know master_secret s.t. H(master_secret || salt) == commitment
              AND HKDF(master_secret, epoch) == ephemeral_id_in_proof"

  Without revealing master_secret

On-chain verification:
  Verifier checks π against:
    - commitment (stored on-chain at registration)
    - ephemeral_id (in submitted proof)
    - epoch (in submitted proof)

  If π is valid → attribute reward to the staking address linked to commitment
  Without learning master_secret or linking encounters across epochs
```

For Midnight integration: Midnight's Compact language supports ZK circuits natively. The HKDF preimage proof is a standard circuit that fits well within Midnight's proof system.

6.4 Encounter Data Minimization

What the network learns per proof submission:

- A proof occurred at epoch E (not exact time)
- Two ephemeral IDs were involved (not persistent identities)
- The approximate signal strength (not precise distance or location)
- Duration bracket (not exact duration)

What the network does NOT learn:

- Real device identities
- Location (no GPS data transmitted)
- Social graph (ephemeral IDs rotate, preventing cross-epoch linkage)

6.5 Relay Node Privacy

Relay nodes route proof submissions. A relay must not be able to learn more than is necessary:

```
Onion-style submission:
  Device → [Encrypt to relay public key] → Relay → [Decrypt, forward] → Network

  OR, simpler for v1:

  Device generates a fresh ephemeral submission keypair per proof batch
  Submits via Tor or Nym mixnet if privacy is paramount
  Relay sees the proof content but not which staking address it belongs to
  Attribution happens on-chain via ZK proof
```

For v1, the simpler approach: relay nodes are not trusted with identity, they are only trusted to forward. On-chain ZK proofs handle attribution.

Part VII: Economic Incentive Design

7.1 Lessons from Helium and Nodle Token Economics

Helium's failure pattern: Fixed emission schedule + rapid device growth = hyperinflation. Network grew faster than demand for the token, causing price collapse, which reduced miner incentives, which reduced network quality. The fix (HIP-51 tokenomics overhaul) came too late.

Nodle's failure pattern: No reward cap per device, no proof-of-work cost, no slashing. Bot farms captured a majority of rewards. Token distribution was highly concentrated in fraudulent actors.

7.2 Recommended Token Emission Structure

```
Total supply: Fixed cap (e.g., 1 billion tokens)
Emission schedule: Halving every 4 years (Bitcoin-style, adjusted for network maturity)

Reward pool allocation:
  60%  →  Encounter rewards (devices proving proximity)
  20%  →  Relay node rewards (infrastructure operators)
  10%  →  Anti-fraud reserve (slash pool + reporter bounties)
  10%  →  Protocol treasury (grants, development, ecosystem)

Dynamic emission dampening:
  base_reward = fixed_schedule_emission / active_device_count
```

```
BUT: active_device_count is measured as unique verified devices in the epoch
(not self-reported count, not staked count — only devices with valid proofs)

This auto-adjusts: more devices = less reward per device = less incentive to farm
```

7.3 Proof-of-Work Cost for Rewards

Require devices to perform a lightweight proof-of-work computation before submitting proofs. This does two things:

1. Adds real computational cost to reward farming (device farms have CPU overhead)
2. Provides a natural rate-limiter without centralized throttling

```
Submission PoW:
  Find nonce such that:
    H(proof_hash || submission_timestamp || nonce) < difficulty_target

  Difficulty calibrated so honest device solves in ~30 seconds on mid-range Android
  (Qualcomm Snapdragon 6xx: ~2M hashes/second SHA256)
  This is negligible for one device; costly for 10,000 in a farm

  Difficulty adjusts every epoch based on submission volume
```

7.4 Encounter Quality Scoring

Not all encounters are equal. Reward encounters based on quality:

```
reward(encounter) = base_reward * quality_multiplier

quality_multiplier components:
  novelty_bonus:      Higher if this device pair has not encountered recently
                      (Prevents farming via repeated encounter with same partner)

  duration_bonus:     Logarithmic scaling with encounter duration
                      (10-second encounter gets 0.3x, 5-minute gets 1.0x, 10+ min gets 1.2x)

  diversity_bonus:    Higher for devices with diverse encounter partners over 30 days
                      (Rewards devices actually moving through diverse social spaces)

  stake_multiplier:   Higher stake tier → slightly higher reward (1.0x to 1.5x)
                      (Incentivizes skin in the game, but not so high it's the only strategy)

  fraud_penalty:      Multiplied by (1 - fraud_score(device))
                      (Fraudulent devices get less even before detection)
```

7.5 Vesting and Lock-Up

To prevent pump-and-dump by early device farmers:

- Rewards earned in epoch E vest linearly over 30 days
- Staked tokens have a 14-day unbonding period
- Relay node rewards vest over 90 days (longer lock for higher rewards)

This delays the payoff for fraud enough that the working capital cost matters.

Part VIII: Relay Node Architecture

8.1 Relay Node Role

Relay nodes are opt-in infrastructure operators who:

- Maintain persistent connectivity to the network
- Accept proof submissions from nearby devices (via local BLE or IP)
- Forward proofs to the blockchain
- Maintain a local Bloom filter of seen session_ids for deduplication
- Optionally serve as BLE proxy for devices with intermittent connectivity

They do NOT:

- Validate proofs cryptographically (this happens on-chain)
- Store user identity data
- Control reward distribution

8.2 Relay Reward Safety

The key risk: relay nodes can attempt to generate fake proofs themselves, or preferentially forward proofs from colluding devices. Mitigations:

Relay staking requirement:
  Relay nodes stake a significant bond (e.g., 10,000 tokens minimum)
  Bond is slashable for forwarding invalid proofs or proved collusion

Relay selection randomization:
  Devices do not choose their relay node
  They submit to a randomly selected relay from the local peer set
  Random selection uses VRF (verifiable random function) with device's local seed
  This prevents a relay from exclusively serving its own fake devices

Relay proof attribution:
  Relay signs the proof batch it submits with its relay identity
  If proofs in that batch are later found fraudulent, relay bears partial liability

Geographic diversity requirement:
  A relay's reward is scaled by the geographic diversity of proofs it forwards
  (measured by approximate BLE signal diversity, not GPS)
  A relay only forwarding proofs from a single cluster gets reduced rewards

Relay uptime measurement:
  A small set of "beacon" proofs are submitted by the protocol treasury
  Beacons measure which relays forward them promptly
  Relay uptime score affects reward multiplier
  Cannot be gamed (relay doesn't know which proofs are beacons)

8.3 Relay Node Technical Stack

Hardware requirements (minimum):
  Any Android device with persistent internet connection
  OR Raspberry Pi 4 / equivalent ARM single-board computer
  OR Cloud VPS (small instance)

Software components:
  - BLE proxy service (Android) or USB BLE dongle driver (SBC/VPS)
  - Proof ingestion API (gRPC/REST)
  - Local deduplication store (LevelDB or RocksDB)
  - Cardano/Midnight submission client
  - Monitoring and health reporting

Network topology:
  Devices ──BLE──→ Relay ──IP──→ Cardano/Midnight node
  Devices ──IP──→ Relay (when on same Wi-Fi, for rapid submission)
  Devices ──IP──→ Direct submission (no relay, lower reward)

Rewarding direct submission at slightly less than relay-forwarded submission incentivizes relay adoption without making it mandatory.

Part IX: Global Scalability Architecture

9.1 Scaling Phases

Phase 1 (0–100K devices): Monolithic proof submission. All proofs go to a single Cardano smart contract endpoint. Collusion detection runs off-chain on a trusted but auditable server. Acceptable for bootstrapping.

Phase 2 (100K–10M devices): Geographic sharding. Divide the world into geographic shards (H3 hexagonal grid at resolution 4 — approximately 1,000 km² cells). Each shard has its own proof accumulator contract. Cross-shard proofs (two devices from different shards encountering each other) go to both shards. Collusion detection runs per-shard with cross-shard correlation.

Phase 3 (10M–1B+ devices): Full decentralization. Proof aggregation using recursive SNARK proofs. Individual device proofs are aggregated off-chain into a single validity proof per epoch per shard. Only the aggregated proof (a few hundred bytes) goes on-chain per epoch per shard. This is the Mina Protocol approach applied to encounter proofs.

9.2 Proof Aggregation

At Phase 3 scale, individual proof submission is infeasible (1B devices × 10 encounters/day × 1KB/proof = 10 PB/day on-chain). Aggregation is mandatory.

```
Proof aggregation pipeline:

Level 0 (device):
  Individual EncounterProof
  Size: ~1 KB per proof

Level 1 (relay aggregate):
  Relay collects N proofs per epoch window
  Generates: SNARK proof that all N proofs are valid
  (Can use recursive proof composition — each proof proves validity of previous batch)
  Size: ~1-10 KB regardless of N

Level 2 (shard aggregate):
  Shard coordinator (elected via VRF) aggregates relay-level proofs
  Generates: Top-level SNARK proof for entire shard's epoch
  Submits single proof on-chain
  Size: ~10 KB for any number of proofs in shard

On-chain:
  Verify Level 2 proof per shard per epoch
  Update reward accumulators
  Detect and reject duplicates via Bloom filter in smart contract state
```

The ZK proof system for this should be Groth16 or PLONK for efficiency. Midnight's native proof system aligns well with this architecture.

9.3 State Management at Scale

```
On-chain state that must be maintained:
  Per device:
    - commitment hash (32 bytes)
    - stake amount (8 bytes)
    - fraud score bucket (1 byte)
    - reward balance (8 bytes)
    - last epoch active (4 bytes)
  = ~53 bytes × 1B devices = ~53 GB (impractical for Cardano UTxO model directly)

Solution: Off-chain state with on-chain commitments
  - Device state lives in a sparse Merkle tree
  - Root of the Merkle tree committed on-chain each epoch
  - Devices prove their state inclusion with Merkle proof in their submission
  - Only the root (32 bytes) and recent updates live on-chain
  - Full state maintained by relay nodes and archive services
```

9.4 BLE Density Management

At high device density (concerts, transit, urban centers), BLE collision becomes a significant problem:

```
Adaptive advertising back-off:
  If device observes >50 advertisers simultaneously:
    → Increase advertising interval to 5000ms
    → Only initiate handshakes with devices not recently encountered
    → Prioritize handshakes with devices that have higher stake (more valuable proofs)

Encounter cap per epoch:
  Maximum 20 unique encounters per 15-minute epoch per device
  Encounters beyond cap are recorded but receive 0 reward (prevents urban farming)

Channel diversity:
  Use all 3 BLE advertising channels (37, 38, 39) in rotation
  Scan across all channels in sequence
  Reduces per-channel collision probability by 3x
```

Part X: Recommended Protocol Architecture (Summary)

10.1 System Layers

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          APPLICATION LAYER                               │
│  Android App: BLE Manager, GATT Handler, Proof Store                     │
│  User: ephemeral_id rotation, stake management, submission               │
└──────────────────────────────────┬───────────────────────────────────────┘
                                   │ BLE + IP
┌──────────────────────────────────▼───────────────────────────────────────┐
│                          RELAY LAYER (Optional)                          │
│  Relay Node: Proof ingestion, deduplication, batch forward               │
│  Staked, accountable, rewarded for uptime and diversity                  │
└──────────────────────────────────┬───────────────────────────────────────┘
                                   │ IP + gRPC
┌──────────────────────────────────▼───────────────────────────────────────┐
│                     AGGREGATION & VALIDATION LAYER                       │
│  Shard Coordinators: Proof collection, SNARK aggregation                 │
│  Fraud Detection: Graph analysis, anomaly scoring                        │
│  Collusion Oracle: Cross-shard fraud correlation                         │
└──────────────────────────────────┬───────────────────────────────────────┘
                                   │ L1 Submission
┌──────────────────────────────────▼───────────────────────────────────────┐
│                          CONSENSUS LAYER                                 │
```

```
│ Cardano L1: Stake registry, reward distribution, slashing  │
│ Midnight: ZK identity attribution, private state management │
└──────────────────────────────────────────────────────────────┘
```

10.2 Technology Choices

Component	Recommended Technology	Rationale
Signing	Ed25519 (Android Keystore)	Hardware-bound, cannot extract key
BLE	Android BLE API, GATT	Native, no custom hardware needed
ZK proofs	Noir (for Midnight), Groth16	Midnight-native, efficient on mobile
Proof aggregation	PLONK recursive proofs	Compact, verifiable
Device attestation	Play Integrity API	Best anti-emulator on Android
Local storage	Room + SQLCipher	Encrypted local proof store
Fraud detection	Off-chain graph DB (Neo4j or TigerGraph)	Graph queries, not suited to L1
On-chain	Cardano + Midnight	Cardano UTxO security + Midnight ZK
Relay communication	gRPC + TLS 1.3	Efficient binary protocol
Hash function	SHA-256 for proofs, Blake2b for ZK	Speed and ZK-friendliness

10.3 Implementation Priority Order

For a proof-of-concept that robustly addresses the core challenges, implement in this order:

1. **GATT handshake with Ed25519 signing** — The encounter proof is the foundation. Get this right first.
2. **Android Keystore key binding** — Without this, all other fraud resistance is weakened.
3. **Epoch-based ephemeral IDs** — Privacy from day one, not bolted on later.
4. **Local proof storage and replay detection** — Build deduplication before building submission.
5. **Adaptive BLE duty cycling** — Battery matters for user retention.
6. **On-chain stake registration and proof submission** — Start with a simple Cardano contract.
7. **Fraud scoring pipeline** — Can be off-chain initially; bring on-chain as trust is established.
8. **Relay node infrastructure** — Add after core proof flow is stable.
9. **ZK reward attribution** — Requires Midnight integration; schedule for v2.
10. **Proof aggregation (recursive SNARK)** — Necessary only at scale >100K devices.

Appendix A: Encounter Proof Pseudocode

```python
```

```python
# Device A initiates handshake with Device B

def initiate_handshake(device_a, device_b_ble_address):
    epoch = current_epoch()
    nonce_a = secrets.token_bytes(32)

    # Send nonce and ephemeral ID to B via GATT WRITE
    gatt_write(device_b_ble_address, {
        'nonce': nonce_a,
        'ephemeral_id': device_a.current_ephemeral_id(epoch),
        'epoch': epoch
    })

    # Receive B's response via GATT NOTIFY
    response = gatt_await_notify(timeout=5.0)
    nonce_b = response['nonce']
    ephemeral_id_b = response['ephemeral_id']

    # Derive session ID
    session_id = sha256(nonce_a + nonce_b + epoch.to_bytes(4))

    # Measure RSSI (B's signal as observed by A)
    rssi_b_to_a = device_a.measure_rssi(device_b_ble_address)

    # Sign: A signs (session_id, A's RSSI observation, B's nonce)
    sig_a = device_a.keystore_sign(sha256(session_id + rssi_b_to_a.to_bytes(1) + nonce_b))

    # Send signature to B
    gatt_write(device_b_ble_address, {'sig': sig_a, 'rssi_observed': rssi_b_to_a})

    # Receive B's signature
    b_response = gatt_await_notify(timeout=5.0)
    sig_b = b_response['sig']
    rssi_a_to_b = b_response['rssi_observed']

    # Assemble proof
    proof = EncounterProof(
        ephemeral_id_A=device_a.current_ephemeral_id(epoch),
        ephemeral_id_B=ephemeral_id_b,
        epoch=epoch,
        nonce_A=nonce_a,
        nonce_B=nonce_b,
        session_id=session_id,
        rssi_A_to_B=rssi_a_to_b,
        rssi_B_to_A=rssi_b_to_a,
        sig_A=sig_a,
```

```
        sig_B=sig_b,
        hardware_attestation=device_a.get_play_integrity_token(session_id)
    )

    # Validate before storing
    assert verify_proof(proof), "Proof verification failed"
    local_proof_store.save(proof)
    return proof

def verify_proof(proof):
    # Recompute session_id
    expected_session = sha256(proof.nonce_A + proof.nonce_B + proof.epoch.to_bytes(4))
    assert expected_session == proof.session_id

    # Verify A's signature
    msg_a = sha256(proof.session_id + proof.rssi_B_to_A.to_bytes(1) + proof.nonce_B)
    assert ed25519_verify(proof.sig_A, msg_a, lookup_pubkey(proof.ephemeral_id_A))

    # Verify B's signature
    msg_b = sha256(proof.session_id + proof.rssi_A_to_B.to_bytes(1) + proof.nonce_A)
    assert ed25519_verify(proof.sig_B, msg_b, lookup_pubkey(proof.ephemeral_id_B))

    # Check RSSI consistency (basic physics check)
    path_loss_A = proof.tx_power_A - proof.rssi_A_to_B
    path_loss_B = proof.tx_power_B - proof.rssi_B_to_A
    assert abs(path_loss_A - path_loss_B) < 15  # dB, allows for antenna asymmetry

    # Check epoch freshness
    assert abs(proof.epoch - current_epoch()) <= MAX_SUBMISSION_DELAY_EPOCHS

    return True
```

Appendix B: Key Risk Register

Risk	Likelihood	Impact	Primary Mitigation
Emulator farms	High	High	Play Integrity API + Keystore binding
Real device farms	Medium	High	Staking cost + reward dampening at scale
Replay attacks	Medium	Medium	Epoch-bound nonces + session_id deduplication
Colluding relay+devices	Medium	High	Random relay assignment + relay stake
Token hyperinflation	High	High	Fixed supply + per-device dampening

Risk	Likelihood	Impact	Primary Mitigation
Battery drain user churn	High	High	Adaptive duty cycling + clear UX
Privacy violations	Low	Very High	Ephemeral IDs + ZK attribution from day 1
Play Integrity unavailability	Medium	Medium	Fallback to reputation scoring
BLE collision in dense areas	Medium	Low	Encounter cap + adaptive advertising
Shard coordinator attack	Low	High	VRF election + multi-party aggregation

---

This document represents a point-in-time engineering analysis. Protocol parameters (epoch length, stake amounts, reward splits, fraud thresholds) should be treated as calibration variables to be tuned based on empirical network data during rollout. No adversarial system design survives first contact with a real adversary unchanged — plan for iteration.